

System Design Document

1. Overview

System Name: The Unified Service Scheduler

Prepared By: Vo Hoang Nguyen

Date: 7/14/2026

Version: 1.0

2. System Objectives

The **Unified Service Scheduler** aims to modernize the appointment booking process for automotive dealerships by replacing manual scheduling with an automated and reliable system. The application ensures that service appointments are only confirmed when the required resources—including a qualified technician and an available service bay—are simultaneously available for the entire service duration.

The system is designed with maintainability, scalability, and reliability in mind. It provides a clear separation of responsibilities between application components, enforces business rules through centralized service logic, and persists appointment data in a relational database. The architecture also considers future enhancements such as multi-dealership support, notification services, and integration with external dealership management systems.

The objectives of the system are:

- Automate the service appointment booking process.
- Prevent scheduling conflicts by validating technician and service bay availability.
- Persist appointment information in a consistent and reliable manner.
- Provide a RESTful API that can be consumed by different client applications.
- Serve as a maintainable and extensible foundation for future dealership scheduling features.

3. Scope

3.1. In Scope

The scope of this project is limited to implementing the backend service for the **Unified Service Scheduler**. The system focuses on the core appointment booking workflow and the validation of resource availability within a dealership.

The implementation includes the following capabilities:

- Create a service appointment for a customer and vehicle.
- Validate the appointment request and business rules.
- Check the availability of both a qualified technician and a service bay for the requested time slot.
- Assign an available technician and service bay to the appointment.
- Persist appointment information in a relational database.
- Expose RESTful APIs for appointment scheduling and availability checking.
- Provide automated tests for the core business logic.

3.2. Out Scope

To keep the implementation focused on the core scheduling functionality, the following features are intentionally excluded:

- User authentication and authorization.
- Customer registration and account management.
- Payment processing.
- Notification services (Email, SMS, Push Notification).
- Calendar synchronization with third-party services.
- Technician shift management.
- Dynamic service duration estimation.
- Reporting and analytics.
- Multi-region or distributed deployment.

4. Assumptions and Constraints

4.1. Assumptions

ID	Assumption	Rationable
A1	An appointment may include one or more service types.	Customers often request multiple services during a single dealership visit.
A2	Each service type has a predefined estimated duration. The total appointment duration is calculated as the sum of the durations of all selected services.	The assessment does not specify how service duration should be determined.
A3	All services within an appointment are performed sequentially by the same technician in the same service bay.	Simplifies scheduling while satisfying the resource availability requirement.
A4	Each appointment belongs to exactly one dealership, and resource allocation is performed only within that dealership.	The assessment requires booking an appointment for a specific dealership.
A5	Technicians are qualified for specific service types based on predefined skills.	The assessment requires assigning a qualified technician

	Only technicians who satisfy all required service skills can be assigned to an appointment.	but does not define the qualification mechanism.
A6	A technician and a service bay cannot be assigned to more than one appointment during overlapping time periods.	Prevents scheduling conflicts and ensures exclusive resource allocation.
A7	Master data (customers, vehicles, dealerships, technicians, service bays, and service types) already exists in the system before appointment scheduling.	Managing master data is outside the scope of this assessment.
A8	When multiple qualified technicians or service bays are available, the system selects the first available resource with the lowest identifier.	Ensures deterministic behavior and simplifies testing.
A9	Appointment times are interpreted using the dealership's local time zone.	Time zone conversion is outside the scope of this assessment.

4.2. Constraints

The implementation is intentionally limited to the core scheduling functionality.

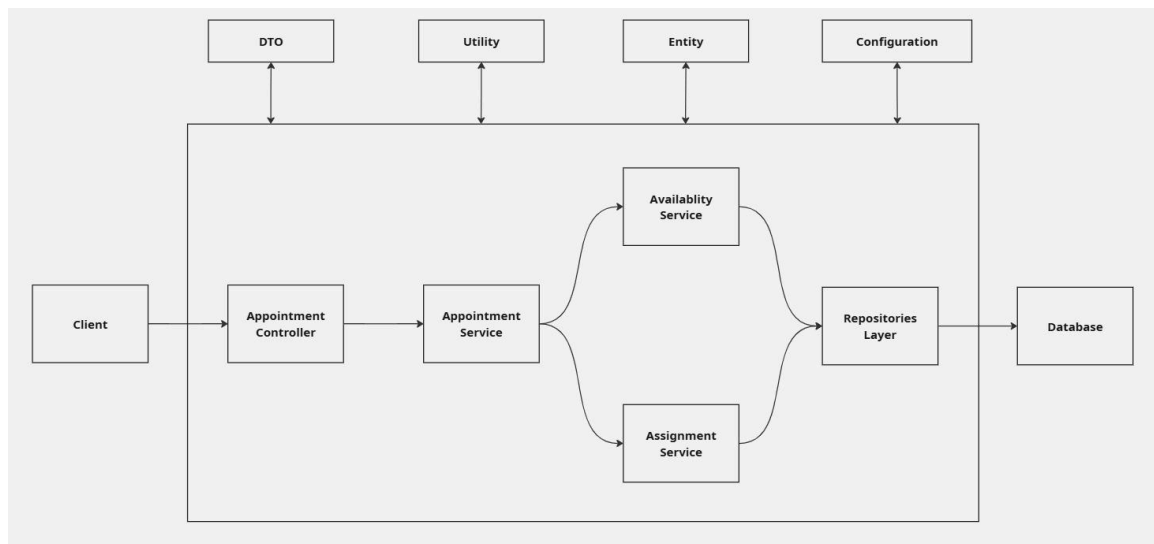
- Authentication and authorization are not implemented.
- Payment processing is not supported.
- Email, SMS, or push notifications are not included.
- Technician shift scheduling and leave management are not supported.
- Dynamic service duration estimation is not supported.
- Integration with external dealership systems is outside the scope of this implementation.
- High availability, distributed deployment, and disaster recovery are considered future enhancements rather than implementation requirements.

5. System Architecture

5.1. Architecture Overview

The Unified Service Scheduler is designed as a layered backend application following the Controller–Service–Repository pattern. The architecture separates API handling, business logic, and data access responsibilities to improve maintainability, testability, and future extensibility.

5.2. Architecture Diagram



5.3. Component Responsibilities

5.3.1. Core Components

Component	Responsibility
AppointmentController	Exposes REST endpoints and handles HTTP requests/responses.
AppointmentService	Orchestrates the appointment booking workflow and manages business transactions.
AvailabilityService	Checks technician and service bay availability for the requested appointment.
Repositories	Provides data access and persistence for all domain entities.
Database	Stores appointments, resources, and master data.

5.3.2. Supporting Components

Component	Responsibility
-----------	----------------

DTO	Defines request and response models for API communication.
Entity	Represents the application's domain model and database mappings.
Configuration	Manages application and framework configurations.
Utility	Provides reusable helper methods shared across the application.

5.4. Architectural Decisions

5.4.1. Layered Architecture

The system uses a layered architecture because it provides:

- Clear separation of concerns.
- Easier unit testing.
- Better maintainability.
- Simpler onboarding for future developers.

5.4.2. Single Database

A single PostgreSQL database is used because the assessment focuses on scheduling logic rather than distributed system concerns

5.4.3. Transaction Boundary

Appointment creation is executed within a database transaction to ensure that:

- An appointment is never partially created.
- Technician and service bay assignments remain consistent.
- Failures roll back all related changes..

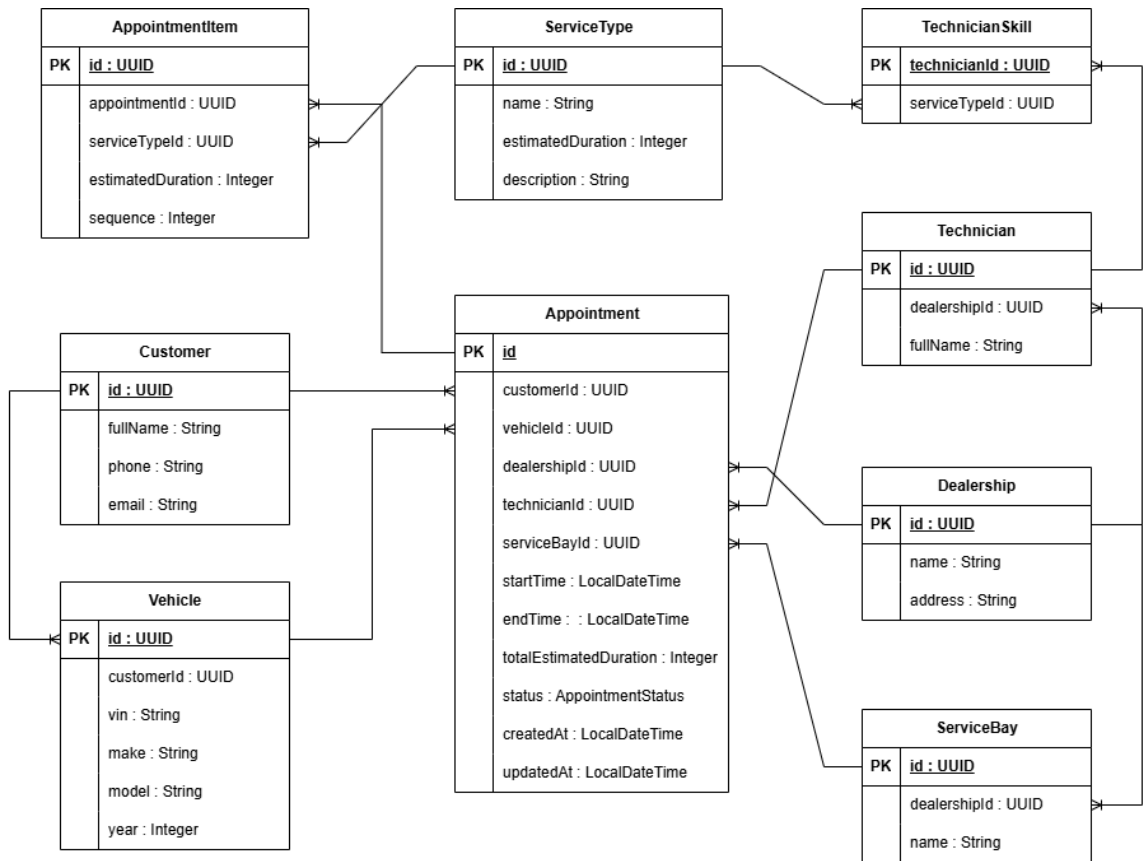
6. Data Model & Database Design

6.1. Overview

The domain model represents the core business entities involved in the dealership appointment scheduling process. It focuses on business relationships rather than

database implementation details, providing a conceptual view of how appointments, resources, and services interact within the system.

6.2. ERD Diagram



6.3. Entity Description

Entity	Description
Appointment	Represents a scheduled service appointment.
AppointmentItem	Represents an individual service within an appointment.
ServiceType	Defines an available service and its estimated duration.
Technician	Represents a qualified technician.
ServiceBay	Represents a physical service bay.

Customer	Represents the customer requesting the appointment.
Vehicle	Represents the customer's vehicle.
Dealership	Represents the dealership where the appointment takes place.

6.4. Design Decisions

Decision	Rationale
Appointment & AppointmentItem Separation	Separates appointment information from requested services, enabling multiple services per appointment while maintaining a normalized database design.
TechnicianSkill Junction Table	Models the many-to-many relationship between technicians and service types, reducing redundancy and simplifying skill validation.
Single Technician & Service Bay Assignment	Each appointment is assigned to one technician and one service bay, simplifying scheduling logic and aligning with the project scope.
Dynamic Availability Calculation	Resource availability is calculated from existing appointments instead of dedicated schedule tables, avoiding duplicated scheduling data.
UUID-based Primary Keys	UUIDs provide globally unique identifiers and support future scalability across distributed systems.
Database Normalization (3NF)	The schema follows Third Normal Form (3NF) to reduce redundancy, improve data consistency, and simplify maintenance.

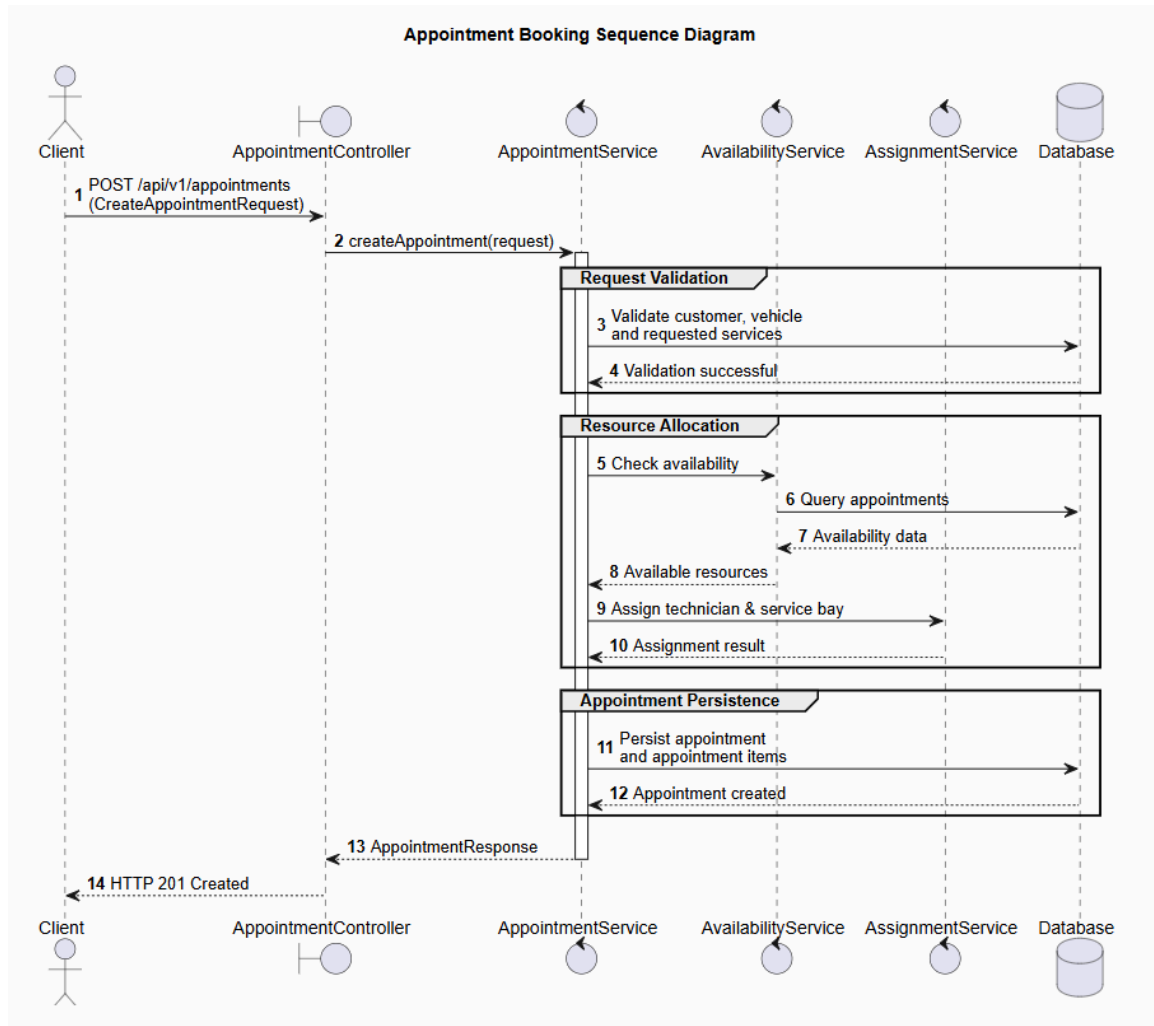
7. Data Flow

7.1. Workflow Overview

The appointment booking workflow begins when a customer submits a booking request containing the dealership, vehicle, preferred start time, and requested services. The

system validates the request, identifies qualified technicians and available service bays, creates the appointment, and returns the booking confirmation.

7.2. Booking Flow



7.3. Workflow Explanation

Step	Component	Description
1	Client	Sends a request to create a new appointment, including customer, vehicle, preferred time, dealership, and requested services.
2	AppointmentController	Receives the request and delegates it to the AppointmentService for processing.

3	AppointmentService	Validates the request by verifying the customer, vehicle, and requested service types.
4	AvailabilityService	Calculates the total appointment duration and retrieves available resources based on the requested time slot.
5	AssignmentService	Selects the most appropriate technician and service bay from the available resources.
6	AppointmentService	Persists the appointment and its associated appointment items in the database.
7	AppointmentController	Returns the created appointment with an HTTP 201 Created response to the client.

8. REST API Design

8.1. API Overview

The system exposes a RESTful API designed around business resources and follows standard REST principles. APIs are organized into two main categories: Appointment APIs, which support the core appointment scheduling workflow, and Master Data APIs, which provide reference information such as customers, vehicles, dealerships, and service types required during appointment creation. All endpoints are versioned using the `/api/v1` prefix to ensure backward compatibility and facilitate future API evolution.

8.2. Appointment APIs

Method	Endpoint	Description
GET	<code>/api/v1/appointments</code>	Retrieve all appointments.
POST	<code>/api/v1/appointments</code>	Create a new appointment.
GET	<code>/api/v1/appointments/{id}</code> .	Retrieve appointment details by ID.

8.3. Customer APIs

Method	Endpoint	Description
GET	/api/v1/customers	Retrieve all customers.
GET	/api/v1/customers/{customerId}/vehicles	Retrieve vehicles owned by a customer.

8.4. Master Data APIs

Method	Endpoint	Description
GET	/api/v1/dealerships	Retrieve available dealerships.
GET	/api/v1/service-types	Retrieve available service types.

8.5. API Design Principles

Principle	Description
RESTful Resource Naming	Resources are exposed using nouns (e.g., /appointments, /customers) instead of verbs.
HTTP Methods	Standard HTTP methods (GET, POST) are used according to REST conventions.
Versioning	APIs are versioned using the /api/v1 prefix to support future evolution.
Consistent Responses	All endpoints return standardized success and error response structures.
Appropriate Status Codes	Standard HTTP status codes (e.g., 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 409 Conflict) are used consistently.

9. Technology Choices

Technology	Purpose	Justification
Java 21	Programming language	Provides modern language features, long-term support (LTS), and excellent performance.
Spring Boot 3	Backend framework	Simplifies REST API development with strong ecosystem support and dependency injection.
Spring Data JPA (Hibernate)	ORM	Reduces boilerplate code and simplifies database interactions.
Microsoft SQL Server	Relational database	Provides ACID transactions, strong consistency, and robust support for relational data and concurrent workloads.
Maven	Build tool	Manages dependencies and automates project builds.
Spring Validation	Input validation	Validates incoming requests using Bean Validation.
OpenAPI / Swagger	API documentation	Validates incoming requests using Bean Validation.
JUNIT 5	Standard unit testing framework for Java.	Standardizes test structure, lifecycle, and assertions.
Mockito	Mocking framework.	Isolates code by simulating external dependencies.
Spring Boot Test	Integration testing support.	Manages the ApplicationContext for full-stack testing.
MockMvc	Spring Web controller testing.	Tests HTTP endpoints without launching a full server.

10. Observability

10.1. Overview

Observability is essential for maintaining system reliability, identifying performance bottlenecks, and troubleshooting production issues. The system should provide visibility into application behavior through logging, metrics, and monitoring.

10.2. Logging

The application will implement structured logging to capture important system events and errors.

Key logging information includes:

- Incoming API requests (HTTP method, endpoint, response status)
- Business events (appointment creation, update, cancellation)
- Validation failures
- Database operation errors
- Unexpected exceptions

10.3. Metrics

The system should expose application metrics to monitor system health and business performance.

Category	Metrics
API Performance	Request count, response time, error rate
Appointment Service	Number of created appointments, cancelled appointments
Database	Query latency, connection pool usage
Infrastructure	CPU, memory usage

10.4. Monitoring and Alerting

A monitoring system can collect metrics and trigger alerts when abnormal behavior occurs.

Example alerts:

- High API error rate (>5%)
- Increased response latency
- Database connection failures

- Application instance unavailable

Possible tools:

- Spring Boot Actuator for application metrics
- Prometheus for metrics collection
- Grafana for visualization and dashboards

11. AI Collaboration

11.1. Overview

AI tools were used during the development process as a productivity assistant while maintaining engineering ownership of the final solution.

AI was mainly utilized for:

- Exploring design alternatives
- Reviewing implementation approaches
- Generating initial code scaffolding
- Identifying potential edge cases
- Improving documentation quality

AI was used to accelerate development and improve solution quality, while all final technical decisions were reviewed and validated by the developer.

11.2 AI Usage Examples

11.2.1. System Design Review

AI was used to support architectural decision-making and evaluate different design approaches.

Main areas of assistance:

- Choosing the appropriate application architecture
- Defining responsibilities between Controller, Service, and Repository layers
- Reviewing entity relationships and data modeling decisions
- Evaluating API design approaches

The final design decisions were reviewed and adjusted based on:

- Business requirements
- Maintainability
- Simplicity
- Long-term scalability

11.2.2. Code Assistance

AI was used to provide implementation guidance and accelerate development.

Examples include:

- Generating initial Spring Boot project structure
- Suggesting entity mapping approaches using JPA/Hibernate
- Reviewing REST API implementation patterns
- Suggesting validation strategies
- Identifying potential code improvements

All generated suggestions were manually reviewed before being integrated into the codebase.

11.2.3. Testing and Edge Case Identification

AI was used to improve testing coverage by identifying scenarios that could potentially be missed.

Examples:

- Creating appointments with invalid references
- Duplicate appointment creation scenarios
- Missing required fields
- Invalid input validation cases
- Database constraint violations
- Unexpected error handling scenarios

These suggestions were evaluated and incorporated into the testing process where appropriate.

11.3. Human Validation

AI-generated suggestions were not applied blindly.

Each recommendation was evaluated based on:

- Requirement correctness
- Code readability
- Security considerations
- Performance impact
- Long-term maintainability

The developer remained responsible for:

- Final architecture decisions
- Implementation choices
- Code quality
- Testing strategy

AI was considered a supporting tool to improve productivity and quality, rather than a replacement for engineering judgment.

